

객체지향 핵심 문법 총정리

오늘의 강의 학습 요약

본 강의에서는 클래스 간 관계(Has-a/Is-a)를 UML과 자바 코드로 연결해 정리한 뒤, 상속의 동작 원리(단일 상속, Object 최상위 클래스, 생성자 호출 순서, super/this)와 오버라이딩 규칙을 체계적으로 확장했습니다. 이어서 toString() 재정의의 의미와 "참조변수 출력 시 자동 호출" 동작을 근거로, 재사용 클래스에서 왜 관례적으로 toString()을 구현하는지까지 연결했습니다.

다음으로 다형성(부모 타입으로 자식 객체 참조)의 핵심 효과(배열/파라미터의 타입 통합)와 함께, 동적 타이핑(동적 바인딩)으로 인해 "컴파일 시점과 실행 시점의 메소드 선택이 달라지는 이유"를 설명했습니다. 다형성에서 자식 전용 멤버 접근을 위한 다운캐스팅, 원하는 타입만 선별하는 instanceof 패턴을 정리했고, 마지막으로 패키지, 임포트와 접근 지정자 규칙을 실습으로 검증한 뒤 추상 클래스의 문법과 목적(하위 클래스 메소드 통일 강제)을 구현 예제로 마무리했습니다.

1. 클래스 관계(Has-a, Is-a)

클래스 관계는 Has-a(전체-부분, 집합/합성)와 Is-a(상속)로 구분되며, UML 표기와 자바 코드 표현이 일관되게 연결됩니다.

Has-a 관계는 한 클래스가 다른 클래스를 "멤버 변수로 포함하여 사용"하는 형태입니다. 트럭이 엔진, 라디오를 부품처럼 포함하는 것처럼 전체(Whole)와 부분(Part)을 이루며, "홀-파트 관계"로도 정리됩니다.

Has-a는 생명주기 관점에서 두 가지로 나뉘며 UML에서는 마름모로 구분됩니다. 채워진 마름모와 비어있는 마름모가 각각 합성과 집합을 의미하며, 실무에서는 "대부분 Has-a가 많다"는 점이 강조됩니다.

자바 코드에서는 포함하려는 타입을 필드로 선언하면 Has-a 관계가 됩니다. 다른 클래스를 "참조한다"는 것은 곧 "사용한다"는 의미이므로, 참조를 위해서는 필드(변수) 보유가 전제가 됩니다.

Is-a 관계는 상속(extends)으로 표현되며, UML에서는 삼각형 화살표(실선)로 나타냅니다. 상속은 부모 구성요소를 자식이 재사용할 수 있게 하며, 재정의(오버라이딩)로 동작을 변경할 수도 있습니다.

구분	핵심 의미	UML 표기 힌트	자바 표현
Has-a(합성/집합)	전체-부분(사용 관계)입니다.	채운/빈 마름모로 구분됩니다.	필드로 다른 클래스를 포함합니다.
Is-a(상속)	부모-자식(확장 관계)입니다.	삼각형 실선 화살표입니다.	extends 키워드를 사용합니다.

핵심 키워드 #Has-a #Is-a #UML 마름모 #홀-파트 #extends

2. 상속의 핵심 규칙과 super/this

자바는 단일 상속을 지원하며, 모든 클래스는 `Object`를 최상위로 두고 생성자 체인과 `super/this` 호출 규칙에 의해 초기화 순서가 결정됩니다.

자바는 단일 상속만 지원하며, 다중 상속을 허용하는 언어(예: 파이썬)와 대비됩니다. 다중 상속은 편리해 보이지만 계층 구조가 불명확해질 수 있어, 자바는 “계층의 명확성”을 택한 설계로 정리됩니다.

`extends` 를 명시하지 않아도 모든 클래스는 자동으로 `extends Object` 가 포함됩니다. 따라서 `Object` 는 모든 클래스의 최상위 클래스이며, 간접 상속 구조를 통해 모든 클래스의 공통 부모가 됩니다.

상속에서 상속되지 않는 요소도 명확히 구분해야 합니다. 생성자와 `private` 멤버는 상속되지 않으며, 그 외 멤버는 접근 범위에 따라 상속, 사용이 가능합니다.

객체 생성 시에는 항상 부모가 먼저 생성됩니다. `new Cat()` 처럼 자식을 생성해도 내부적으로는 `Object` → 부모 → 자식 순으로 생성자 체인이 동작하며, 이는 “부모를 먼저 만들어야 부모 요소를 사용할 수 있다”는 원리에 기반합니다.

생성자 호출 방식은 세 가지로 정리됩니다. 외부에서 생성자를 호출할 때는 `new` , 같은 클래스의 다른 생성자를 호출 때는 `this()` , 부모 생성자를 호출할 때는 `super()` 를 사용합니다.

명시적으로 `super(...)` 또는 `this(...)` 를 호출하는 경우, 반드시 생성자 첫 줄에 위치해야 합니다. 이는 부모가 선언한 인스턴스 변수를 부모가 초기화하는 흐름을 강제하기 위한 규칙으로 설명됩니다.

호출 키워드	호출 대상	사용 위치	핵심 제약
<code>new</code>	외부 클래스의 생성자 호출입니다.	객체 생성 시점입니다.	생성자 호출의 시작점이 됩니다.
<code>this()</code>	같은 클래스의 다른 생성자 호출입니다.	생성자 내부입니다.	첫 줄에만 올 수 있습니다.
<code>super()</code>	부모 클래스 생성자 호출입니다.	생성자 내부입니다.	첫 줄에만 올 수 있습니다.

핵심 키워드

#단일 상속

#Object

#super()

#this()

#생성자 체인

3. 오버라이딩과 toString 재정의

오버라이딩은 부모 메소드를 자식이 동일 시그니처로 재정의하는 규칙이며, toString은 참조변수 출력 시 자동 호출되므로 재정의가 실무적으로 매우 유용합니다.

오버라이딩은 “부모 메소드를 그대로 쓰거나, 필요에 의해 자식이 재정의하는 것”으로 정리됩니다. 재정의된 메소드는 부모와 동일한 메소드 시그니처를 가져야 하며, 접근 지정자는 더 넓게만 변경할 수 있습니다.

오버라이딩 규칙은 다음 조건을 만족해야 합니다. 리턴 타입, 메소드 이름, 파라미터가 동일해야 하며, 접근 지정자는 같거나 더 개방적인 방향이어야 합니다.

`@Override` 어노테이션은 관례적으로 붙이며, 규칙 위반 시 컴파일 에러로 검증을 돕습니다. 즉 “오버라이딩 의도 표기”와 “규칙 자동 검증”을 동시에 제공하는 장치입니다.

오버라이딩이 불가능한 대표 조건도 함께 정리됩니다. `private` 메소드는 상속 자체가 불가하고, `static` 및 `final` 메소드는 재정의가 불가능합니다.

`super` 는 부모를 가리키는 참조이며, (1) 부모 생성자 호출 `super(...)` , (2) 부모 멤버 호출 `super.member` 두 경우에 사용됩니다. 다만 `super.member` 는 부모, 자식에 동일 이름 멤버가 있을 때만 주로 쓰여 빈도는 낮다고 정리됩니다.

`Object` 의 `toString()` 은 문자열을 반환하며 기본 구현은 클래스 정보와 해시(16진수 형태 등)를 포함한 “주소 기반 표현”으로 나타냅니다. 참조변수를 `print` 하면 내부적으로 `toString()` 이 자동 호출되므로, `toString()` 을 재정의하면 출력 편의성이 크게 향상됩니다.

`String` 클래스에서 골뱅이(기본 `Object` 스타일)가 출력되지 않는 이유는 `String` 이 `toString()` 을 이미 재정의하여 “실제 문자열 값”을 반환하기 때문입니다. 따라서 사용자 정의 클래스도 `toString()` 을 재정의하면 별도 `getValue()` 같은 메소드 없이 출력만으로 값을 확인할 수 있습니다.

항목	기본 동작	재정의 효과	실무 관례
참조변수 출력	자동으로 <code>toString()</code> 이 호출됩니다.	원하는 값 형태로 출력이 바뀝니다.	재사용 클래스에서 <code>toString()</code> 재정의가 권장됩니다.
<code>Object</code> <code>toString()</code>	주소 기반 문자열이 반환됩니다.	클래스 의미 표현에 부적합할 수 있습니다.	기본 구현을 그대로 쓰면 골뱅이 형태가 흔히 보입니다.
<code>String</code> <code>toString()</code>	실제 문자열 값이 반환됩니다.	출력이 직관적입니다.	자주 쓰이므로 표준 라이브러리에서 재정의되어 있습니다.

핵심 키워드

#오버라이딩

#Override

#super

#Object

#toString()

4. 다형성, 동적 바인딩, 형변환/instanceof

다형성은 부모 타입으로 자식 객체를 참조하는 능력이며, 동적 바인딩으로 인해 실행 시점에는 실제 객체의 오버라이드 메소드가 호출됩니다.

다형성은 "부모 타입 참조변수 하나가 여러 자식 인스턴스를 참조할 수 있는 능력"입니다. 상속이 전제되며, 자식 타입을 부모 타입으로 참조하는 것은 가능하지만 그 역은 기본적으로 불가능하다고 정리됩니다.

다형성 활용은 크게 배열과 메소드 파라미터로 정리됩니다. 같은 타입만 담는 배열도 부모 타입 배열로 만들면 자식들을 함께 저장할 수 있고, 파라미터도 부모 타입 하나로 다양한 자식 인자를 받을 수 있어 오버로딩 필요가 줄어듭니다.

동적 타이핑은 강의 문맥에서 동적 바인딩과 동일 개념으로 설명됩니다. 컴파일 시점에는 참조변수 타입(부모) 기준으로 메소드 존재 여부를 확인하고, 실행 시점에는 실제 생성된 객체(자식)의 오버라이딩 메소드가 실행됩니다.

동적 바인딩은 "부모에도 있고 자식에도 동일 메소드가 있을 때" 다형성 상태에서 특히 중요합니다. 예를 들어 `Employee e = new Manager(); e.a();` 는 컴파일 시점에는 `Employee`의 `a()` 로 확인되지만, 실행 시점에는 `Manager`의 `a()` 가 실행됩니다.

다형성 상태에서 자식에만 추가된 메소드/변수는 그대로 접근할 수 없습니다. 컴파일러는 부모 타입에서 멤버를 찾기 때문에, 자식 전용 멤버에 접근하려면 다운캐스팅(자식 타입으로 형변환)이 필요합니다.

원하는 자식 타입만 필터링하려면 `instanceof` 를 사용합니다. 배열이나 파라미터로 여러 자식이 섞여 들어온 경우, `instanceof` 로 타입을 확인한 뒤 다운캐스팅하여 자식 전용 메소드를 호출하는 패턴이 핵심입니다.

```
for (Employee emp : employees) {
    if (emp instanceof Engineer) {
        ((Engineer) emp).c(); // 자식 전용 메소드 호출은 캐스팅이 필요합니다.
    }
}
```

상황	컴파일 시점 관점	실행 시점 관점	필요한 처리
부모/자식에 동일 메소드가 있음	부모 타입에서 메소드 존재를 확인합니다.	실제 객체(자식)의 오버라이드 메소드가 실행됩니다.	동적 바인딩이 발생합니다.
자식에만 메소드가 추가됨	부모 타입에 없으면 컴파일 에러가 발생합니다.	실행까지 도달하지 못합니다.	다운캐스팅이 필요합니다.
혼합 배열/파라미터 처리	모두 부모 타입으로 순회됩니다.	실제 인스턴스 타입이 섞여 있습니다.	<code>instanceof</code> 로 필터링 후 캐스팅합니다.

핵심 키워드

#다형성

#동적 바인딩

#다운캐스팅

#instanceof

#오버로딩 대체

5. 패키지/임포트와 접근 지정자

패키지는 클래스 묶음(네임스페이스)이며, 다른 패키지 클래스 접근에는 `import` 또는 정규명 사용이 필요하고 접근 지정자는 패키지/상속 여부에 따라 범위가 달라집니다.

패키지는 관련 클래스 파일을 묶어 관리하는 단위로, 윈도우 폴더 구조와 유사하게 이해할 수 있습니다. 패키지 선언은 파일 상단에 한 번만 가능하며, 서브 패키지로 깊이를 깊게 만들 수도 있지만 과도한 깊이는 관리성을 떨어뜨립니다.

패키지명은 충돌 방지를 위해 유일해야 하며, 관례적으로 도메인을 거꾸로 사용합니다. 예를 들어 `google.com` 은 `com.google` 형태로 패키지명을 구성합니다.

패키지를 명시하지 않으면 “디폴트 패키지”에 속합니다. 따라서 “패키지가 없는 클래스”는 없으며, 명시적 패키지 선언이 없을 때 디폴트 패키지로 분류된다고 정리됩니다.

패키지가 다르면 기본적으로 클래스 접근이 되지 않으며, 이를 해결하는 방법이 `import` 입니다. 또한 `java.lang` 패키지는 예외적으로 `import` 없이도 사용할 수 있어 `String`, `System` 등을 바로 쓰는 이유가 설명됩니다.

다른 패키지 클래스를 쓰는 방식은 두 가지입니다. (1) 정규명(패키지명.클래스명)을 코드에서 직접 쓰는 방식, (2) `import` 후 클래스명만 쓰는 방식이며, 일반적으로는 `import` 사용이 권장됩니다.

IDE에서는 자동 임포트가 가능하며, 클래스명 자동완성(예: `Ctrl+Space`)로 선택하면 `import` 구문이 자동 삽입되는 흐름이 정리됩니다.

접근 지정자는 `public`, `protected`, (default), `private` 네 가지이며, “같은 패키지인지”, “상속 관계인지”를 함께 고려해야 정확히 판단됩니다. 같은 클래스 내부에서는 모두 접근 가능하고, 같은 패키지의 다른 클래스에서는 `private` 만 불가하며, 다른 패키지에서는 기본적으로 `public` 만 가능하되 상속 관계에서는 `protected` 까지 가능하다고 실습으로 검증합니다.

접근 범위	같은 클래스	같은 패키지(다른 클래스)	다른 패키지(상속 아님)	다른 패키지(상속)
<code>public</code>	가능합니다.	가능합니다.	가능합니다.	가능합니다.
<code>protected</code>	가능합니다.	가능합니다.	불가합니다.	가능합니다.
(default)	가능합니다.	가능합니다.	불가합니다.	불가합니다.
<code>private</code>	가능합니다.	불가합니다.	불가합니다.	불가합니다.

핵심 키워드

`#package`

`#import`

`#java.lang`

`#접근 지정자`

`#protected`

6. 추상 클래스와 추상 메소드의 목적

추상 클래스는 미완성 클래스이며 직접 객체 생성이 불가하고, 하위 클래스에서 메소드를 통일, 강제하기 위해 추상 메소드 재정의의를 요구합니다.

추상 클래스는 선언 시 `abstract class` 형태로 정의되며, 일반 클래스(콘크리트 클래스)와 구분됩니다. 콘크리트 클래스는 변수, 생성자, (콘크리트)메소드로 구성되며, 추상 클래스는 여기에 “추상 메소드”를 추가로 가질 수 있습니다.

콘크리트 메소드는 구현부(중괄호)가 있어 동작이 확정된 반면, 추상 메소드는 `abstract` 가 붙고 구현부 없이 세미콜론으로 끝나 동작이 미정입니다. 이 “미정(구체화되지 않음)” 때문에 추상 클래스는 미완성 클래스가 되며, 직접 `new` 로 객체 생성이 불가능합니다.

추상 클래스를 사용하려면 상속을 통해 자식(콘크리트) 클래스를 만들고, 자식이 추상 메소드를 반드시 오버라이딩하여 완성해야 합니다. 이때 추상 메소드 재정의는 강제이며, 누락 시 컴파일 에러가 발생합니다.

추상 클래스는 추상 메소드를 반드시 가져야 하는 것은 아니지만, 추상 메소드가 하나라도 있으면 그 클래스는 반드시 추상 클래스여야 합니다. 또한 추상 메소드에서는 `abstract` 키워드를 생략할 수 없으며, 이는 인터페이스와의 차이로 별도 메모가 필요하다고 정리됩니다.

추상 클래스도 상속 계층의 “부모 타입”이므로 다형성을 적용할 수 있습니다. 즉 추상 클래스 타입 변수로 자식(콘크리트) 객체를 참조하는 형태가 가능하며, 이를 통해 공통 계약(메소드 통일)을 유지하면서 객체를 관리할 수 있습니다.

추상 클래스의 핵심 용도는 하위 클래스 메소드의 “통일성과 강제성”입니다. 오버라이딩만으로는 재정의가 선택 사항이라 누락될 수 있는데, 추상 메소드를 두면 자식이 반드시 구현해야 하므로 메소드 이름, 시그니처를 강제로 일치시킬 수 있습니다.

방법	통일 방식	강제성	누락 시 결과
일반 오버라이딩	부모 메소드를 재정의해 통일합니다.	약합니다.	누락해도 컴파일이 통과될 수 있습니다.
추상 메소드	자식이 반드시 구현하도록 강제합니다.	강합니다.	누락 즉시 컴파일 에러가 발생합니다.

핵심 키워드

#abstract class

#abstract method

#미완성 클래스

#강제성

#메소드 통일